Q1A | Summarize the Data

> A. Summarize the data. How much data is present? What attributes/features are continuous valued? Which attributes are categorical?

This notebook answers the question in three parts:

1. **How much data is present?** Row and column counts, the train/test split, duplicates, and missing values.
2. **Which attributes are continuous?** Data types, distinct values, and value ranges.
3. **Which attributes are categorical?** The conclusion that follows from part 2.

IMPORTANT

Tables describe `train` and `test` combined unless they are labeled by dataset. The notebook only counts and describes the data, and no modeling decisions are made here.

Answer Summary

- **How much data is present?** 1,893 rows and 9 columns (1 target, `Life Ladder`, and 8 features), split 82.36% train (1,559 rows) and 17.64% test (334 rows). There are no duplicate rows. 283 values are missing, all in train, across 185 rows (11.87% of train rows).
- **Which attributes are continuous?** All 9. Every column is `float64` with 372 to 1,519 distinct values, no value repeats more than 17 times, and none is whole-number only.
- **Which attributes are categorical?** None. No column holds labels or a small set of repeated codes.

Flags:

- **Missing values:** test is complete, so handling them only changes training rows. `Perceptions of corruption` (86) and `Generosity` (71) hold more than half of them, the target has none, and 5 rows are missing 6 of their 8 features.
- **Scale:** feature stds run from 0.085 to 7.546, which matters for any scale-sensitive step.
- **Sign:** `Generosity` is the only column with negative values (min -0.335).

Imports

In [1]:
```
import pandas as pd

import Q1.consts as Q1_CONSTS

from utils import read_csv_to_df
```

Fetch Data

In [2]:
```
train_df = read_csv_to_df(path=Q1_CONSTS.TRAIN_PATH)
test_df = read_csv_to_df(path=Q1_CONSTS.TEST_PATH)
combined_df = pd.concat([train_df, test_df], ignore_index=True)
```

1. How Much Data is Present?

In [3]:
```
summary = pd.DataFrame(
        {
                "row count": [train_df.shape[0], test_df.shape[0],
combined_df.shape[0]],
                "% row count": [
                        round(train_df.shape[0] / combined_df.shape[0] *
100, 2),
                        round(test_df.shape[0] / combined_df.shape[0] *
100, 2),
                        round(combined_df.shape[0] / combined_df.shape[0] *
100, 2),
                ],
                "columns count": [train_df.shape[1], test_df.shape[1],
combined_df.shape[1]],
                "missing values": [train_df.isnull().sum().sum(),
test_df.isnull().sum().sum(), combined_df.isnull().sum().sum()],
                "rows w/ missing values": [
                        train_df.isnull().any(axis=1).sum(),
                        test_df.isnull().any(axis=1).sum(),
                        combined_df.isnull().any(axis=1).sum()
                ],
                "% rows w/ missing values": [
                        round(train_df.isnull().any(axis=1).sum() /
train_df.shape[0] * 100, 2),
                        round(test_df.isnull().any(axis=1).sum() /
test_df.shape[0] * 100, 2),
                        round(combined_df.isnull().any(axis=1).sum() /
combined_df.shape[0] * 100, 2),
                ]
        },
        index=["train_df", "test_df", "combined_df"],
        dtype=object,  # keeps the counts as whole numbers after the
transpose
```

```
)

summary.T
```

Out[3]:

	train_df	test_df	combined_df
row count	1559	334	1893
% row count	82.36	17.64	100.0
columns count	9	9	9
missing values	283	0	283
rows w/ missing values	185	0	185
% rows w/ missing values	11.87	0.0	9.77

- The 9 columns are 1 target (`Life Ladder`) and 8 features. There is no column to group or order rows by.
- All 283 missing values are in train (185 rows, 11.87% of train rows). Test is complete, so handling missing values only changes training rows.

Duplicate Rows

In [4]:

```
duplicates = pd.DataFrame(
        {
                "duplicate rows": [
                        train_df.duplicated().sum(),
                        test_df.duplicated().sum(),
                        combined_df.duplicated().sum(),
                ],
        },
        index=["train_df", "test_df", "combined_df"],
)

duplicates
```

Out[4]:

	duplicate rows
train_df	0
test_df	0
combined_df	0

- No duplicate rows in train, in test, or in the two combined, so no row is shared between train and test.

Missing Values by Feature

In [5]:

```
missing_by_feature = pd.DataFrame({
        'Missing (train)': train_df.isna().sum(),
        '% of rows (train)': (train_df.isna().sum() / len(train_df) *
```

```
100).round(2),
}).sort_values('Missing (train)', ascending=False)

missing_by_feature
```

Out[5]:

	Missing (train)	% of rows (train)
Perceptions of corruption	86	5.52
Generosity	71	4.55
Healthy life expectancy at birth	37	2.37
Log GDP per capita	26	1.67
Freedom to make life choices	22	1.41
Positive affect	17	1.09
Negative affect	14	0.90
Social support	10	0.64
Life Ladder	0	0.00

- `Perceptions of corruption` (86) and `Generosity` (71) hold more than half of the 283 missing values, so how missing values are handled matters most for these two features.
- No feature is missing in more than 5.52% of train rows, and the target is never missing, so no row lacks a label.

Missing Values per Row

In [6]:

```
missing_per_row = train_df.isna().sum(axis=1).value_counts().sort_index()

missing_per_row_df = pd.DataFrame({
        "train rows": missing_per_row,
        "% of train rows": (missing_per_row / len(train_df) *
100).round(2),
})
missing_per_row_df.index.name = "missing values in row"

missing_per_row_df
```

Out[6]:

missing values in row	train rows	% of train rows
0	1374	88.13
1	130	8.34
2	27	1.73
3	23	1.48
6	5	0.32

- Using only complete rows would leave 1,374 train rows (88.13%), yet 130 of the 185 incomplete rows are missing just 1 value.

- 5 rows are missing 6 of their 8 features, so they carry almost no feature information.

2. Which Attributes are Continuous?

An attribute is continuous if it measures a quantity that can take any value within a range. The stored data type alone does not settle that (categories can be stored as numbers), so the evidence used is the number of distinct values, how often values repeat, and whether values are fractional and spread across a range.

In [7]:
```
info_df = pd.DataFrame({
        "dtype": combined_df.dtypes,
        "unique values": combined_df.nunique(),
        "most frequent value count": combined_df.apply(lambda col:
col.value_counts().iloc[0]),
}).sort_values(by=["unique values", "most frequent value count"],
ascending=False)

info_df
```

Out[7]:

	dtype	unique values	most frequent value count
Life Ladder	float64	1519	5
Log GDP per capita	float64	1476	4
Healthy life expectancy at birth	float64	825	15
Generosity	float64	602	13
Perceptions of corruption	float64	566	14
Freedom to make life choices	float64	533	10
Social support	float64	452	15
Positive affect	float64	430	12
Negative affect	float64	372	17

- All 9 columns are `float64` with 372 to 1,519 distinct values, and no value repeats more than 17 times, so no column behaves like a set of codes or labels.

Value Ranges

In [8]:
```
value_ranges = pd.DataFrame({
        "min": combined_df.min(),
        "max": combined_df.max(),
        "mean": combined_df.mean().round(3),
        "std": combined_df.std().round(3),
        "whole numbers only": combined_df.apply(lambda col: (col.dropna() %
1 == 0).all()),
}, index=combined_df.columns)

value_ranges
```

Out[8]:

	min	max	mean	std	whole numbers only
Life Ladder	2.375	8.019	5.461	1.124	False
Log GDP per capita	6.635	11.648	9.356	1.156	False
Social support	0.290	0.987	0.812	0.119	False
Healthy life expectancy at birth	32.300	77.100	63.337	7.546	False
Freedom to make life choices	0.258	0.985	0.741	0.142	False
Generosity	-0.335	0.698	-0.000	0.162	False
Perceptions of corruption	0.035	0.983	0.748	0.187	False
Positive affect	0.322	0.944	0.709	0.107	False
Negative affect	0.083	0.705	0.269	0.085	False

- Feature scales are far apart (std from 0.085 to 7.546), which matters for any scale-sensitive step (SGD, regularization).
- `Generosity` is the only column with negative values (min -0.335), which matters for any transform that needs positive inputs.

All 9 attributes (the target and all 8 features) are continuous. None is whole-number only, and each takes many distinct values across a range.

3. Which Attributes are Categorical?

No attributes are categorical. All 9 columns are continuous, so no column holds labels or codes and there is nothing to encode.